

MODULE 14

Networking & Persistence

Retrofit, Coroutines, Room, Secure Storage, and Offline Sync

Phase 03 | Android Development with Kotlin

What You Will Build in This Module

1

Networking

Retrofit, kotlinx.serialization, coroutines, interceptors, MockWebServer

2

Persistence and Security

Room, migrations, DataStore, Keystore encryption, token refresh, pinning

3

Offline-First

Single source of truth, outbox, WorkManager sync, capstone



Section 1

Networking: Retrofit, JSON, and Coroutines

- 1.1 HTTP, REST, and Retrofit Foundations
- 1.2 JSON Serialization with `kotlinx.serialization`
- 1.3 Coroutines for Asynchronous Networking
- 1.4 Interceptors, Error Handling, and Testing the Network Layer

HTTP, REST, and Retrofit Foundations

- Read an HTTP request and response and name its verb, path, headers, status code, and body
- Configure Retrofit and OkHttp in a Gradle project with the correct permission and base URL
- Declare a typed API interface using Retrofit annotations and suspend functions



Anatomy of a Mobile API Call

- Verb says intent:
 - GET reads
 - POST creates
 - PUT replaces
 - DELETE removes
- Status code says outcome:
 - 2xx success
 - 4xx client fault
 - 5xx server fault
- Headers carry identity and format:
Authorization, Content-Type, Accept
- Always HTTPS; the transport protects the user, not your code

Code Bite

```
GET /v1/accounts HTTP/1.1
Host: api.pncmobile.example
Authorization: Bearer eyJhbGciOi...
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: private, max-age=30
```

```
[{"id":"acc_1","name":"Virtual
Wallet","masked_number":"****4821","balance"
:2450.75}]
```



Retrofit and OkHttp: Division of Labor

OkHttp moves bytes: connections, TLS, timeouts, interceptors, cache

Retrofit turns an annotated interface into OkHttp calls

A converter turns JSON bodies into Kotlin objects and back

One shared client per app, never one per request



Gradle Setup

- Retrofit 3.0, OkHttp 5.5, kotlin.serialization converter (already included earlier)
- Serialization plugin applied in the module build file
- INTERNET permission is declared, never requested at runtime
- Version catalog keeps versions in one place

Code Bite

```
plugins {  
    alias(libs.plugins.kotlin.serialization)  
}  
dependencies {  
    implementation(libs.retrofit)  
    implementation(libs.okhttp)  
    implementation(libs.okhttp.logging)  
}
```



Gradle Permissions

- The INTERNET permission MUST be included in AndroidManifest.xml
- If not, the requests will just silently fail
 - Which may cause errors downstream in your app, of course!

Code Bite

```
// AndroidManifest.xml
<uses-permission android:name =
    "android.permission.INTERNET" />
```



Declaring the API Interface

- One annotation per HTTP verb: @GET, @POST, @PUT, @DELETE
- @Path fills the URL, @Query appends parameters, @Body sends JSON
- Mark functions suspend so callers use coroutines
- Return the body type, or Response<T> when you need the status code

Code Bite

```
interface AccountsApi {  
    @GET("accounts")  
        suspend fun getAccounts():  
        List<AccountDto>  
  
    @GET("accounts/{id}/transactions")  
        suspend fun getTransactions(  
            @Path("id") accountId: String,  
            @Query("limit") limit: Int = 50,  
            @Query("cursor") cursor: String? = null  
        ): TransactionPageDto  
}
```



Sending Data

- @Path fills the URL, @Query appends parameters, @Body sends JSON
- Mark functions suspend so callers use coroutines
- Return the body type, or Response<T> when you need the status code

Code Bite

```
interface AccountsApi {  
    . . .  
  
    @POST("transfer")  
    suspend fun transfer(@Body request:  
        TransferRequestDto): Response<Unit>  
  
}
```



Building the Client and Reaching Localhost

- Base URL must end with a slash
- Emulator reaches your computer at 10.0.2.2, not localhost
- Allow cleartext only in a debug network security config
- Release builds stay HTTPS only

Code Bite

```
val json = Json { ignoreUnknownKeys = true }

val retrofit = Retrofit.Builder()
    .baseUrl(BuildConfig.API_BASE_URL)
    .client(okHttpClient)
    .addConverterFactory(
        json.asConverterFactory(
            "application/json".toMediaType()
        )
    ).build()

val accountsApi: AccountsApi =
    retrofit.create(AccountsApi::class.java)
```



Lesson 1.1

Skill Check ✓

1. A response returns status 401. Which layer of the app should react first?

- A) The Room database
- B) The authentication layer, which should attempt a token refresh
- C) The Compose theme
- D) The Gradle build

2. Why does the emulator need 10.0.2.2 instead of localhost?

- A) It is faster
- B) localhost on the emulator points to the emulator itself
- C) Android forbids localhost for security
- D) Retrofit rewrites it

3. Which configuration belongs on OkHttpClient rather than Retrofit?

- A) @GET paths
- B) Read timeout
- C) Return types
- D) Query parameter names

4. Where should cleartext HTTP be allowed?

- A) In the main manifest for all builds
- B) Only in a debug-only network security config
- C) In every build type
- D) Nowhere, ever, including debug



JSON Serialization with `kotlinx.serialization`

- Explain why network DTOs are separate from domain models
- Represent money and timestamps safely
- Annotate DTOs with `@Serializable` and `@SerialName` and configure `Json` for forward compatibility
- Handle unknown enum values without crashing



DTO, Entity, Domain: Three Different Shapes

DTO mirrors the server JSON exactly

Domain model mirrors what the app needs

Entity mirrors a database row (later lesson)

Mappers are the only code that knows two shapes at once



DTO, Entity, Domain: Three Different Shapes (code)

```
@Serializable
```

```
data class AccountDto(  
    val id: String,  
    val name: String,  
    @SerializedName("masked_number") val maskedNumber: String,  
    @SerializedName("balance_cents") val balanceCents: Long  
)
```

```
data class Account(  
    val id: String,  
    val name: String,  
    val maskedNumber: String,  
    val balance: Money  
)
```

```
fun AccountDto.toDomain() = Account(id, name, maskedNumber, Money(balanceCents))
```



Json Configuration

- Defaults make fields optional in the payload
- ignoreUnknownKeys lets the server add fields safely

Code Bite

```
val appJson = Json {  
    ignoreUnknownKeys = true  
    explicitNulls = false  
    coerceInputValues = true  
    encodeDefaults = false  
}
```



Annotations for Json Encoding

- `@Serializable` generates the serializer at compile time
- `@SerializedName` maps snake_case JSON to camelCase Kotlin

Code Bite

```
@Serializable
data class TransferRequestDto(
    @SerializedName("from_account")
    val fromAccountId: String,
    @SerializedName("to_account")
    val toAccountId: String,
    @SerializedName("amount_cents")
    val amountCents: Long,
    val memo: String? = null
)
```



Money Done Safely

- Never use Double for currency
 - $0.1 + 0.2$ is not 0.3
- Safe (accurate) options include:
 - Integer minor units (cents)
 - BigDecimal strings
- It can be useful to wrap either of those options in a class

Code Bite

```
class Money(val cents: Long) {  
  
    operator fun plus(o: Money) =  
        Money(cents + o.cents)  
  
    fun format(  
        locale: Locale = Locale.getDefault()  
    ): String =  
        NumberFormat  
            .getCurrencyInstance(locale)  
            .format(cents / 100.0)  
  
}
```



Money and Time Done Safely

- Parse ISO-8601 timestamps to Instant with a custom serializer
- Format for display only at the UI edge, using the user's locale

Code Bite

```
object InstantSerializer:  
    KSerializer<Instant> {  
    override val descriptor =  
        PrimitiveSerialDescriptor("Instant",  
            PrimitiveKind.STRING)  
    override fun serialize(encoder: Encoder,  
        value: Instant) =  
        encoder.encodeString(value.toString())  
    override fun deserialize(  
        decoder: Decoder  
    ): Instant =  
        Instant.parse(decoder.decodeString())  
}
```



Enums, Sealed Types, and Unknown Values

A new server enum value must not crash an old app

`coerceInputValues` maps unknown enums to the declared default

Add an explicit UNKNOWN case and render it gracefully

Polymorphic payloads use a sealed class with a discriminator



Enums, Sealed Types, and Unknown Values (code)

```
@Serializable
```

```
enum class TxType {  
    @SerializedName("debit") DEBIT,  
    @SerializedName("credit") CREDIT,  
    @SerializedName("hold") HOLD,  
    UNKNOWN  
}
```

```
@Serializable
```

```
data class TransactionDto(  
    val id: String,  
    @SerializedName("amount_cents") val amountCents: Long,  
    val type: TxType = TxType.UNKNOWN,  
    @Serializable(with = InstantSerializer::class) val posted: Instant  
)
```



Lesson 1.2

Skill Check ✓

1. Why not let Composables read DTOs directly?

- A) DTOs are slower
- B) Wire format changes would ripple through the UI
- C) Compose cannot display DTOs
- D) DTOs cannot be null

2. What does `ignoreUnknownKeys = true` protect against?

- A) Missing required fields
- B) New fields added by the server
- C) Slow networks
- D) Expired tokens

3. Which type is safest for currency amounts?

- A) Float
- B) Double
- C) Long cents or BigDecimal
- D) String concatenation

4. How do you survive a new enum value from the server?

- A) Catch the exception in the Composable
- B) Declare an UNKNOWN default and enable `coerceInputValues`
- C) Delete the enum
- D) Ask the server team never to change it



Section 2

Persistence and Security: Room, DataStore, Keystore, Auth

- 2.1 Room Fundamentals: Entities, DAOs, Database
- 2.2 Relations, Transactions, and Migrations
- 2.3 DataStore and Secure Storage with the Android Keystore
- 2.4 Authentication Plumbing: Tokens, Refresh, Pinning, Logout

Room Fundamentals: Entities, DAOs, Database

- Choose Room over raw SQLite and explain its three building blocks
- Write DAOs with suspend queries, reactive Flow queries, and @Upsert
- Define entities with primary keys, indexes, and type converters
- Test a DAO against an in-memory database



Why Room, and Its Three Parts

- Entity: a class that maps to a table
- DAO: an interface of queries, verified at compile time
- Database: the holder that wires entities and DAOs
- Room checks your SQL during the build, not in production

Code Bite

```
// build.gradle.kts
plugins { alias(libs.plugins.ksp) }
dependencies {
    implementation(libs.room.runtime)
    implementation(libs.room.ktx)
    ksp(libs.room.compiler)
    testImplementation(libs.room.testing)
}

ksp { arg("room.schemaLocation",
"$projectDir/schemas") }
```



Entities

- @Entity with a primary key; use string ids from the server
- @ColumnInfo renames columns; @Index speeds lookups

Code Bite

```
@Entity(tableName = "accounts")
data class AccountEntity(
    @PrimaryKey val id: String,
    val name: String,
    @ColumnInfo(name = "masked_number") val
maskedNumber: String,
    @ColumnInfo(name = "balance_cents") val
balanceCents: Long,
    @ColumnInfo(name = "updated_at") val
updatedAtEpochMs: Long
)
```



Type Converters

- SQLite stores primitives only: convert Instant and enums
- Keep entities flat and boring; richer types belong in the domain

Code Bite

```
class Converters {  
    @TypeConverter fun fromInstant(v:  
Instant?): Long? = v?.toEpochMilli()  
    @TypeConverter fun toInstant(v: Long?):  
Instant? = v?.let(Instant::ofEpochMilli)  
}
```



DAOs: Queries, Flow, and Upsert

suspend functions for one-shot reads and writes

Flow return type makes a query observable

@Upsert inserts or updates in one call

Room runs suspend and Flow queries off the main thread



DAOs: Queries, Flow, and Upsert (code)

@Dao

```
interface AccountDao {  
    @Query("SELECT * FROM accounts ORDER BY name")  
    fun observeAccounts(): Flow<List<AccountEntity>>  
  
    @Query("SELECT * FROM accounts WHERE id = :id")  
    suspend fun getById(id: String): AccountEntity?  
  
    @Upsert  
    suspend fun upsertAll(accounts: List<AccountEntity>)  
  
    @Query("DELETE FROM accounts")  
    suspend fun clear()  
}
```



The Database Class and In-Memory Tests

@Database lists entities, version, and
exportSchema = true

Build one instance per process with
Room.databaseBuilder

Tests use Room.inMemoryDatabaseBuilder: fast
and isolated

Never call allowMainThreadQueries in
production



The Database Class

```
@Database(entities = [AccountEntity::class], version = 1, exportSchema = true)
@TypeConverters(Converters::class)
abstract class PncDatabase : RoomDatabase() {
    abstract fun accountDao(): AccountDao
}
```



Lesson 2.1

Skill Check ✓

1. What is the main advantage of Room over raw SQLite?

- A) It is a different database engine
- B) Compile-time SQL verification and less boilerplate
- C) It works without a device
- D) It is encrypted by default

2. What does a DAO method returning Flow do?

- A) Runs once
- B) Re-emits whenever the queried table changes
- C) Blocks the main thread
- D) Syncs to the server

3. Why store money as Long cents in an entity?

- A) SQLite cannot store text
- B) Avoids floating point error and migrates easily
- C) It is required by Room
- D) It makes queries slower

4. Why use an in-memory database in tests?

- A) It persists across runs
- B) It is fast and isolated
- C) It is encrypted
- D) It needs the network



Relations, Transactions, and Migrations

- Model one-to-many relationships with foreign keys and @Relation
- Write and test a manual migration
- Group multi-step writes in a transaction
- Explain why destructive migration is unacceptable for banking data



Foreign Keys and One-to-Many Relations

Declare a ForeignKey so the database enforces integrity

CASCADE deletes children when a parent is removed

A POJO with @Embedded and @Relation loads parent and children together

Room does not auto-join; it runs a second query for relations



Establishing Foreign Keys

```
@Entity(  
    tableName = "transactions",  
    foreignKeys = [  
        ForeignKey(  
            entity = AccountEntity::class,  
            parentColumns = ["id"], childColumns = ["accountId"],  
            onDelete = ForeignKey.CASCADE  
        )  
    ],  
    indices = [Index("accountId")]  
)  
data class TransactionEntity(  
    /* columns */  
    val accountId: String)
```



Using One-to-Many Relationships

```
data class AccountWithTransactions(  
    @Embedded val account: AccountEntity,  
  
    @Relation(parentColumn = "id", entityColumn = "accountId")  
    val transactions: List<TransactionEntity>  
)
```



Transactions: All or Nothing

- @Transaction on DAO methods makes reads consistent
- Partial writes corrupt a sync: use one transaction per refresh
- Keep transactions short; never call the network inside one

Code Bite

```
@Dao
interface AccountDao {
    @Transaction
    @Query("SELECT * FROM accounts WHERE id = :id")
    fun observeWithTransactions(id: String):
        Flow<AccountWithTransactions?>
}
```



Transactions: All or Nothing

- db.withTransaction groups several DAO calls atomically

Code Bite

```
suspend fun replaceAll(  
    db: PncDatabase,  
    accounts: List<AccountEntity>  
) = db.withTransaction {  
    db.accountDao().upsertAll(accounts)  
    db.accountDao()  
        .deleteMissing(accounts.map { it.id })  
  
    }
```



Migrations

Every schema change increments the version

A Migration object describes how to get from version N to N+1

Export schemas to source control so migrations can be tested

`fallbackToDestructiveMigration` deletes user data: forbidden here



Setting up Migrations

```
@Database(  
    entities = [AccountEntity::class, TransactionEntity::class],  
    version = 2,  
    autoMigrations = [  
        AutoMigration(from = 1, to = 2)  
    ],  
    exportSchema = true  
)  
abstract class PncDatabase : RoomDatabase() {  
    /* daos */  
}
```



Migrations (generated code)

```
val MIGRATION_2_3 = object : Migration(2, 3) {  
    override fun migrate(db: SupportSQLiteDatabase) {  
        db.execSQL("ALTER TABLE accounts ADD COLUMN nickname TEXT")  
        db.execSQL("UPDATE accounts SET nickname = name")  
    }  
}
```

```
Room.databaseBuilder(ctx, PncDatabase::class.java, "pnc.db")  
    .addMigrations(MIGRATION_2_3)  
    .build()
```



Testing Migrations

MigrationTestHelper opens a database at the old version

Insert old-shape data, run the migration, validate the new schema

One test per migration, plus one test for the full chain

Commit exported schema JSON files to source control

Lesson 2.2

Skill Check ✓

1. What does ForeignKey.CASCADE on delete do?

- A) Blocks the delete
- B) Deletes child rows when the parent is deleted
- C) Ignores the delete
- D) Copies rows

2. Why wrap replace-all in one transaction?

- A) It is faster to type
- B) A crash mid-way cannot leave half-updated data
- C) Room requires it
- D) It avoids indexes

3. Why is fallbackToDestructiveMigration unacceptable here?

- A) It is slow
- B) It silently deletes user data, including queued transfers
- C) It needs the network
- D) It is deprecated

4. What should a migration test assert beyond schema validity?

- A) Nothing
- B) That existing data survived
- C) App size
- D) Gradle duration

